

ESP32-S3-Touch-AMOLED-1.8

From Waveshare Wiki
Jump to: navigation, search

Overview

[\[Expand\]](#)

Effect Demonstration



(/wiki/File:200px-ESP32-S3-Touch-

AMOLED-1.8-demo-25.png)



(/wiki/File:181px-ESP32-S3-Touch-AMOLED-1.8-demo-28.png)

ESP32-S3 Touch AMOLED
1.8



368×448, UART/USB/GPIO



(/wiki/File:267px-ESP32-S3-Touch-AMOLED-1.8-demo-



24.png)

(/wiki/File:199px-ESP32-S3-Touch-AMOLED-1.8-demo-

29.png)

Usage Instructions

ESP32-S3-Touch-AMOLED-1.8 currently provides two development tools and frameworks, **Arduino IDE** and **ESP-IDF**, providing flexible development options, you can choose the right

development tool according to your project needs and personal habits.

Development Tools

Arduino IDE



(/wiki/File:180px-Arduino-IDE-logo.jpg)

Arduino IDE is an open source electronic prototyping platform, convenient and flexible, easy to get started. After a simple learning, you can start to develop quickly. At the same time, Arduino has a large global user community, providing an abundance of open source code, project examples and tutorials, as well as rich library resources, encapsulating complex functions, allowing developers to quickly implement various functions.

ESP-IDF



(/wiki/File:180px-ESP-IDF-logo.jpg)

ESP-IDF, or full name Espressif IDE, is a professional development framework introduced by Espressif Technology for the ESP series chips. It is developed using the C language, including a compiler, debugger, and flashing tool, etc., and can be developed via the command lines or through an integrated development environment (such as Visual Studio Code with the Espressif IDF plugin). The plugin offers features such as code navigation, project management, and debugging, etc..

Each of these two development approaches has its own advantages, and developers can choose according to their needs and skill levels. Arduino are suitable for beginners and non-professionals because they are easy to learn and quick to get started. ESP-IDF is a better choice for developers with a professional background or high performance requirements, as it provides more advanced development tools and greater control capabilities for the development of complex projects.

Components Preparation

- ESP32-S3-Touch-AMOLED-1.8 x1
- TF card x 1
- USB cable (Type-A to Type-C) x 1



(/wiki/File:400px-ESP32-S3-Touch-

AMOLED-1.8-uesbotes-01.png)

Before operating, it is recommended to browse the table of contents to quickly understand the document structure. For smooth operation, please read the FAQ carefully to understand possible problems in advance. All resources in the document are provided with hyperlinks for easy download.

Working with Arduino

This chapter introduces setting up the Arduino environment, including the Arduino IDE, management of ESP32 boards, installation of related libraries, program compilation and downloading, as well as testing demos. It aims to help users master the development board and facilitate secondary development.



(/wiki/File:Arduino-flow-04.png)

Environment Setup

Download and Install Arduino IDE

- Click to visit the Arduino official website (<https://www.arduino.cc/en/software>), select the corresponding system and system bit to download



Arduino IDE 2.3.3

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits
Windows MSI installer
Windows ZIP file

Linux AppImage 64 bits (X86-64)
Linux ZIP file 64 bits (X86-64)

macOS Intel, 10.15: "Catalina" or newer, 64 bits
macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

(/wiki/File:ESP32-S3-AMOLED-1.91-Ar-software-01.png)

- Run the installer and install all by default

The environment setup is carried out on the Windows 10 system, Linux and Mac users can access Arduino-esp32 environment setup (<https://docs.espressif.com/projects/arduino-esp32/en/latest/installing.html>) for reference

Install ESP32 development board

- Before using ESP32-related motherboards with the Arduino IDE, you must first install the software package for the **esp32 by Espressif Systems** development board
- According to **Board installation requirement**, it is generally recommended to use **Install Online**. If online installation fails, use **Install Offline**.
- For the installation tutorial, please refer to Arduino board manager tutorial ()
- ESP32-S3-Touch-AMOLED-1.8 required development board installation description

Board name	Board installation requirement	Version number requirement
esp32 by Espressif Systems	"Install Offline" / "Install Online"	≥3.0.6

Install Library

- When installing Arduino libraries, there are usually two ways to choose from: **Install online** and **Install offline**. **If the library installation requires offline installation, you must use the provided library file**
For most libraries, users can easily search and install them through the online library manager of the Arduino software. However, some open-source libraries or custom libraries are not synchronized to the Arduino Library Manager, so they cannot be acquired through online searches. In this case, users can only manually install these libraries offline.
- ESP32-S3-Touch-AMOLED-1.8 library file is stored in the sample program, click here to jump:ESP32-S3-Touch-AMOLED-1.8 Demo
- For library installation tutorial, please refer to Arduino library manager tutorial ()

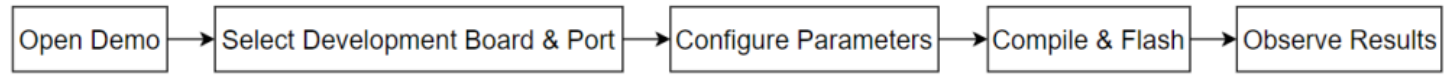
Library Name	Description	Version	Library Installation Requirement
Arduino_DriveBus	CST816 Touch chip driver library	—	"Install Offline"
GFX_Library_for_Arduino	GFX graphical library for SH8601	v1.4.9	"Install Offline"
ESP32_IO_Expander	TCA9554 expansion chip driver library	v0.0.3	"Install Online" or "Install Offline"
lvgl	LVGL graphical library	v8.4.0	"Install Online" requires copying the demos folder to src after installation. "Install Offline" is recommended
SensorLib	PCF85063, QMI8658 sensor driver library	v0.2.1	"Install Online" or "Install Offline"
XPowersLib	XP2101 power management chip driver library	v0.2.6	"Install Online" or "Install Offline"
ui_a	Custom ui library	—	"Install Offline"
ui_b	Custom ui library	—	"Install Offline"
ui_c	Custom ui library	—	"Install Offline"
Mylibrary	Development board pin macro definition	—	"Install Offline"
lv_conf.h	LVGL configuration file	—	"Install Offline"

Run the First Arduino Demo

If you are just getting started with ESP32 and Arduino, and you don't know how to create, compile, flash, and run Arduino ESP32 programs, then please expand and take a look. Hope it can help you!

[Expand]

Demos



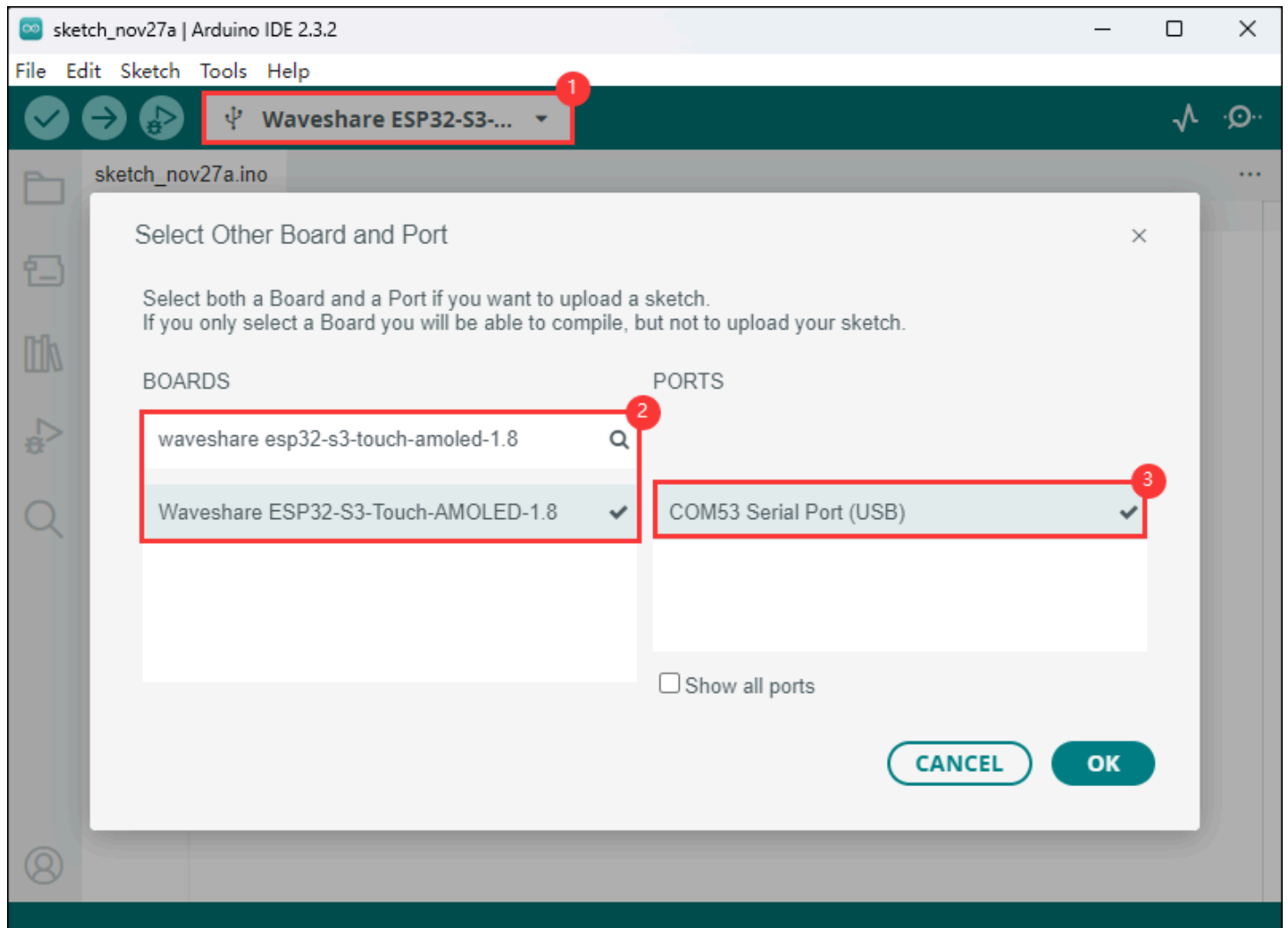
(/wiki/File:Demo-flow-01.png)

■ ESP32-S3-Touch-AMOLED-1.8 Demo

Demo	Basic Description	Dependency Library
01_HelloWorld	Demonstrates the basic graphics library functions and can also be used to test the basic performance of display screens and the display effect of random text	GFX_Library_for_Arduino
02_Drawing_board	Demonstrates the basic graphics library functions and can also be used to test the basic performance of display screens and the display effect of random text	GFX_Library_for_Arduino, Arduino_DriveBus, ESP32_IO_Expander
03_GFX_AsciiTable	Prints ASCII characters in rows and columns on the display screen according to the screen size	GFX_Library_for_Arduino
04_GFX_FT3168_Image	Displays the image effect, change the displayed image through touch	GFX_Library_for_Arduino, Arduino_DriveBus, ESP32_IO_Expander
05_GFX_PCF85063_simpleTime	GFX library displays the current time	SensorLib, GFX_Library_for_Arduino
06_GFX_ESPWiFiAnalyzer	Draws the WiFi band signal strength on the screen	GFX_Library_for_Arduino
07_GFX_Clock	Implement clock through basic pointer marking and time management	GFX_Library_for_Arduino
08_LVGL_Animation	Custom UI, controls backlight brightness	LVGL, Arduino_DriveBus, ui_a
09_LVGL_change_background	Custom UI, controls background color	LVGL, Arduino_DriveBus, ui_b
10_LVGL_PCF85063_simpleTime	LVGL library displays the current time	LVGL, SensorLib
11_LVGL_QMI8658_ui	LVGL draws acceleration line chart	LVGL, SensorLib
12_LVGL_AXP2101_ADC_Data	LVGL displays PMIC data	LVGL, XPowerLib
13_LVGL_Widgets	LVGL demonstration	LVGL, Arduino_DriveBus, ESP32_IO_Expander
14_LVGL_SD_Test	LVGL displays the contents of TF card files	LVGL
15_ES8311	ES8311 driver demo, playing simple audio	——

16_LVGL_Sqprj	squareline UI combined with LVGL demo	LVGL
---------------	---------------------------------------	------

■ ESP32-S3-Touch-AMOLED-1.8 supports direct model selection



(/wiki/File:ESP32-S3-Touch-AMOLED-1.8-demo-01.png)

01_HelloWorld

Demo description

- This demo demonstrates how to control the **SH8601** display using the Arduino GFX library, demonstrating basic graphics library functionality with dynamically changing text. This code can also be used to test the basic performance of the display and the effect of displaying random text

Hardware connection

- Connect the development board to the computer

Code analysis

- Initialize the display:

```
if (!gfx->begin()) {  
  USBSerial.println("gfx->begin() failed!");  
}
```

- Clear the screen and display text:

```
gfx->fillScreen(BLACK);  
gfx->setCursor(10, 10);  
gfx->setTextColor(RED);  
gfx->println("Hello World!");
```

- GIF display:

```
gfx->setCursor(random(gfx->width()), random(gfx->height()));  
gfx->setTextColor(random(0xffff), random(0xffff));  
gfx->setTextSize(random(6), random(6), random(2));  
gfx->println("Hello World!");
```

Result demonstration



(/wiki/File:200px-ESP32-S3-Touch-AMOLED-1.8_241015_23.png)

02_Drawing_board

Demo description

- This example demonstrates how to use the ESP32 to control the FT3168 touch controller and TCA9554 GPIO expander via the I2C interface, while using the Arduino GFX and

Hardware connection

- Connect the development board to the computer

Code analysis

- Screen initialization and brightness gradient animation:

```
gfx->begin();
gfx->fillScreen(WHITE);
for (int i = 0; i <= 255; i++) {
    gfx->Display_Brightness(i);
    gfx->setCursor(30, 150);
    gfx->setTextColor(BLUE);
    gfx->setTextSize(4);
    gfx->println("Loading board");
    delay(3);
}
```

- Touch interrupt handling and coordinate reading

```
void Arduino_IIC_Touch_Interrupt(void) {
    FT3168->IIC_Interrupt_Flag = true;
}

int32_t touchX = FT3168->IIC_Read_Device_Value(FT3168->Arduino_IIC_Touch::Value_Information::TOUCH_COORDINATE_X);
int32_t touchY = FT3168->IIC_Read_Device_Value(FT3168->Arduino_IIC_Touch::Value_Information::TOUCH_COORDINATE_Y);

if (FT3168->IIC_Interrupt_Flag == true) {
    FT3168->IIC_Interrupt_Flag = false;
    USBSerial.printf("Touch X:%d Y:%d\n", touchX, touchY);
    if (touchX > 20 && touchY > 20) {
        gfx->fillCircle(touchX, touchY, 5, BLUE);
    }
}
```

Result demonstration



(/wiki/File:200px-ESP32-S3-Touch-AMOLED-1.8_241015_21.png)

03_GFX_AsciiTable

Demo description

- This demo shows how to display a basic ASCII character table on the SH8601 display by using the Arduino GFX library on an ESP32. The core function of the code is to initialize the display and print ASCII characters by rows and columns on the display according to the screen size

Hardware connection

- Connect the development board to the computer

Code analysis

- Create data bus and graphic display objects
 - Here a data bus object for communicating with the display is created, which is initialized with a specific pin configuration. Then a graphical display object gfx is created, passing in parameters such as data bus, reset pin, rotation angle, whether it is an IPS screen, and the width and height of the display

```
Arduino_DataBus *bus = new Arduino_ESP32QSPI(  
    LCD_CS /* CS */, LCD_SCLK /* SCK */, LCD_SDI00 /* SDI00 */, LCD_SDI01 /* SDI01 */,  
    LCD_SDI02 /* SDI02 */, LCD_SDI03 /* SDI03 */);  
  
Arduino_GFX *gfx = new Arduino_SH8601(bus, -1 /* RST */,  
    0 /* rotation */, false /* IPS */, LCD_WIDTH, LCD_  
HEIGHT);
```

- Draw row and column numbers and character table

- First set the text color to green and print the line numbers one by one on the display. Then set the text color to blue and print the column number. Next, use a loop to draw each character individually to form the character table, with each character using white foreground and black background

```
gfx->setTextColor(GREEN);
for (int x = 0; x < numRows; x++) {
    gfx->setCursor(10 + x * 8, 2);
    gfx->print(x, 16);
}
gfx->setTextColor(BLUE);
for (int y = 0; y < numCols; y++) {
    gfx->setCursor(2, 12 + y * 10);
    gfx->print(y, 16);
}

char c = 0;
for (int y = 0; y < numRows; y++) {
    for (int x = 0; x < numCols; x++) {
        gfx->drawChar(10 + x * 8, 12 + y * 10, c++, WHITE, BLACK);
    }
}
```

Result demonstration



(/wiki/File:200px-ESP32-S3-Touch-AMOLED-1.8_241015_22.png)

04_GFX_FT3168_Image

Demo description

- This demo implements a touch screen display interface using the SH8601 display driven by QSPI on ESP32-S3. It integrates an I2C touch controller (FT3168) for detecting touch input, as well as an I2C GPIO expander (TCA9554) for managing additional output pins.

This code sets the display screen, touch controller, and expander, and cycles through multiple images when touching the screen

Hardware connection

- Connect the development board to the computer

Code analysis

- The image shows:

```
if (fingers_number > 0) {
    switch (Image_Flag) {
        case 0: gfx->draw16bitRGBBitmap(0, 0, (uint16_t *)gImage_1, LCD_WIDTH, LCD_HEIGHT);
        break;
        case 1: gfx->draw16bitRGBBitmap(0, 0, (uint16_t *)gImage_2, LCD_WIDTH, LCD_HEIGHT);
        break;
        case 2: gfx->draw16bitRGBBitmap(0, 0, (uint16_t *)gImage_3, LCD_WIDTH, LCD_HEIGHT);
        break;
        case 3: gfx->draw16bitRGBBitmap(0, 0, (uint16_t *)gImage_4, LCD_WIDTH, LCD_HEIGHT);
        break;
        case 4: gfx->draw16bitRGBBitmap(0, 0, (uint16_t *)gImage_5, LCD_WIDTH, LCD_HEIGHT);
        break;
    }
    Image_Flag++;
    if (Image_Flag > 4) {
        Image_Flag = 0;
    }
}
```

Code modification

- The image content resources are large and need to be set **Tools -> Partition Scheme -> 16M Flash (3MB APP/9.9MB FATFS)**

Result demonstration



(/wiki/File:200px-ESP32-S3-Touch-AMOLED-1.8-demo-03.png)



(/wiki/File:200px-ESP32-S3-Touch-AMOLED-1.8-demo-04.png)



(/wiki/File:200px-ESP32-S3-Touch-AMOLED-1.8-demo-05.png)

05_GFX_PCF85063_simpleTime

Demo description

- This demo demonstrates how to use the PCF85063 RTC module to display the current time on a SH8601 display, retrieving the time every second and updating the display only when the time changes

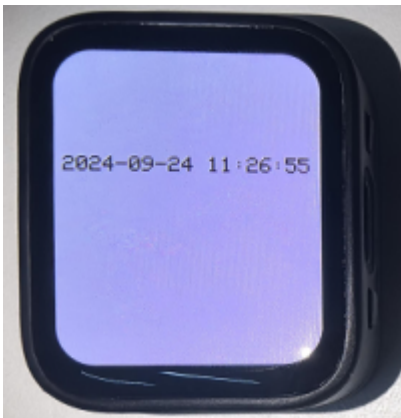
Hardware connection

- Connect the development board to the computer

Code analysis

- **setup:** Initialize the program
 - Serial port initialization to provide a channel for outputting error messages
 - Initialize the real-time clock chip, including connection check and setting the initial time, to ensure the accuracy of the time
 - Initialize the graphic display device, set the background color and brightness, and provide a visual interface for the time display
- **loop:** Continuously check for time changes and update the time display on the display while the program is running
 - Regularly check whether the time has changed, and determine whether the time display needs to be updated by comparing the difference between the current time and the last update time
 - Obtain the time information of the real-time clock and format it so that it is displayed correctly on the display
 - If the time changes, clear the last time display area, set the text color and size, calculate the center position, and display the new time on the display. Finally save the current time as the last time for comparison next time

Result demonstration



(/wiki/File:200px-ESP32-S3-Touch-AMOLED-1.8_241015_20.png)

06_GFX_ESPWiFiAnalyzer

Demo description

- This demo is an example of drawing Wi-Fi band signal strength on the SH8601 display, implementing the function of a Wi-Fi analyzer

Hardware connection

- Connect the development board to the computer

Code analysis

- **setup:** Prepare for the initialization of the entire program
 - Initializes serial port communication and sets the baud rate to 115200 for outputting information and debugging
 - Set WiFi to site mode and disconnect in preparation for subsequent network scans
 - Perform some additional pre-initialization operations based on different hardware conditions (if `GFX_EXTRA_PRE_INIT` is defined)
 - Initialize the graphic display device, calculate some display-related parameters, such as text size, banner height, etc., and set the display brightness. Finally draw the initial banner information
- **loop:** The main program logic of Lines, including performing WiFi network scans, processing scan results, drawing charts, displaying statistics, and performing energy-saving operations as needed
 - WiFi scanning and processing: Perform WiFi network scanning, obtain the number of networks and various network information; Count the number of access points for each channel, calculate the noise level and determine the signal peak; Plot WiFi signal strength charts based on the scan results, including drawing ellipses to represent signal strength and display relevant network information
 - Display statistical information: Print the number of networks found and channel information with less noise on the display screen, draw the baseline and channel markers for the chart, and show the number of access points for each channel
 - Energy-saving operation: Depending on the configured number of scans, perform energy-saving operations when conditions are met, such as turning off the display power pin and entering deep sleep mode

Result demonstration



(/wiki/File:200px-ESP32-S3-Touch-AMOLED-1.8-demo-06.png)

07_GFX_Clock

Demo description

- This demo demonstrates a simple SH8601 clock example, which implements a clock example with a simple marker pointer and time management

Hardware connection

- Connect the development board to the computer

Code analysis

- Draw hour, minute and second hands:

```
void redraw_hands_cached_draw_and_erase() {
    gfx->startWrite();
    draw_and_erase_cached_line(center, center, nsx, nsy, SECOND_COLOR, cached_points, sH
andLen + 1, false, false);
    draw_and_erase_cached_line(center, center, nhx, nhy, HOUR_COLOR, cached_points + ((s
HandLen + 1) * 2), hHandLen + 1, true, false);
    draw_and_erase_cached_line(center, center, nmx, nmy, MINUTE_COLOR, cached_points +
((sHandLen + 1 + hHandLen + 1) * 2), mHandLen + 1, true, true);
    gfx->endWrite();
}
```

Result demonstration



(/wiki/File:200px-ESP32-S3-Touch-AMOLED-1.8-demo-07.png)

08_LVGL_Animation

Demo description

- This demo demonstrates a simple LVGL slider example, where the screen backlight brightness can be changed by altering the slider value

Hardware connection

- Connect the development board to the computer

Code analysis

- Modify the backlight brightness in real time

```
int32_t slider_value = lv_slider_get_value(ui_Slider1);  
int32_t brightness = map(slider_value, 0, 100, 5, 255);  
gfx->Display_Brightness(brightness);
```

Result demonstration



(/wiki/File:200px-ESP32-S3-Touch-AMOLED-1.8-demo-08.png)

09_LVGL_change_background

Demo description

- This demo demonstrates an LVGL backlight brightness change scenario, achieving changes in background color through the definition of simple button components

Hardware connection

- Connect the development board to the computer

Code analysis

- **setup()**: Responsible for initializing the entire system
 - First initialize the serial port for possible debugging output
 - Then initialize the I2C bus and the expansion chip, set the pin mode and initial state of the expansion chip
 - Next keep trying to initialize the touch controller, and set its power mode when it succeeds
 - Initialize the graphics display device and set the brightness, and also print the versions of LVGL and Arduino
 - Initialize LVGL, including registering the debugging print callback function, initializing the display driver, and input device driver
 - Create and start the LVGL timer, initialize the user interface, and print the setup complete message
- **loop**
 - Keep calling `lv_timer_handler()` in the main loop to let the LVGL graphics library handle various tasks
 - Add a small delay to avoid overoccupying CPU resources

- **my_touchpad_read**: Read the touchpad coordinates, set the input device status of LVGL according to the touch state, and update the touch coordinates

Result demonstration



(/wiki/File:200px-ESP32-S3-Touch-AMOLED-1.8-demo-09.png)



(/wiki/File:202px-ESP32-S3-Touch-AMOLED-1.8-demo-10.png)

10_LVGL_PCF85063_simpleTime

Demo description

- This demo demonstrates how to use the PCF85063 RTC module under LVGL to display the current time on a SH8601 display screen, retrieving the time every second and only updating the display when the time changes, which is better than time refresh

Hardware connection

- Connect the development board to the computer

Code analysis

- **setup**: Responsible for initializing various hardware devices and LVGL graphics library environment
 - Serial port initialization: `USBSerial1.begin(115200)` prepares for serial port debugging
 - Real-time clock initialization: Try to initialize the real-time clock `rtc`, enter a loop if it fails. Set the date and time
 - Touch controller initialization: Keep trying to initialize the touch controller `FT3168`, if initialization fails, print an error message and wait with a delay, and print a success message if successful
 - Graphical display initialization: Initialize the graphical display device `gfx`, set the brightness, and print the version information of LVGL and Arduino. Next, initialize the LVGL, including registering the print callback function for debugging, initializing the display driver and the input device driver. Create and start the LVGL timer, finally create a label and set the initial text to "Initializing..."
- **loop**
 - `lv_timer_handler()`: This is an important function in the LVGL graphics library, which is used to handle various timer events, animation updates, input processing and other tasks of the graphical interface. Calling this function in each loop ensures smooth operation of the graphical interface and timely response to interactions
 - Time update and display: Get the time of the real-time clock every second and print it out through the serial port. Then format the time as a string and update the text of the label to display the current time. At the same time, set the font of the label to a specific font. Finally add a small delay

Result demonstration



(/wiki/File:200px-ESP32-S3-Touch-AMOLED-1.8-demo-11.png)

11_LVGL_QMI8658_ui

Demo description

- This demo demonstrates the use of LVGL for graphical display, communicating with the QMI8658 IMU to obtain accelerometer and gyroscope data

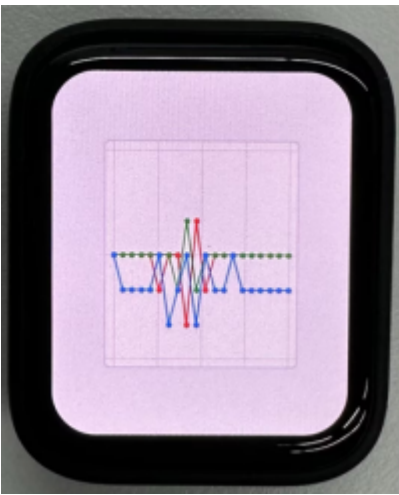
Hardware connection

- Connect the development board to the computer

Code analysis

- **setup**: Responsible for initializing various hardware devices and LVGL graphics library environment
 - Serial port initialization: `USBSerial1.begin(115200)` prepares for serial port debugging
 - Touch controller initialization: Keep trying to initialize the touch controller `FT3168`, if initialization fails, print an error message and wait with a delay, and print a success message if successful
 - Graphical display initialization: Initialize the graphical display device `gfx`, set the brightness, and print the version information of LVGL and Arduino. Next, initialize the LVGL, including registering the print callback function for debugging, initializing the display driver and the input device driver. Create and start the LVGL timer, finally create a label and set the initial text to "Initializing..."
 - Create a chart: Create a chart object `chart`, set the chart type, range, number of data points and other properties of the chart, and add data series for the three axes of acceleration
 - Acceleration sensor initialization: Initialize acceleration sensor `qmi`, configure accelerometer and gyroscope parameters, enable them, and print chip ID and control register information
- **loop**
 - `lv_timer_handler()`: This is an important function in the LVGL graphics library, which is used to handle various timer events, animation updates, input processing and other tasks of the graphical interface. Calling this function in each loop ensures smooth operation of the graphical interface and timely response to interactions
 - Read acceleration sensor data: If the acceleration sensor data is ready, read the acceleration data and print it via the serial port, while updating the chart to display the acceleration data. If the gyroscope data is ready, read the gyroscope data and print it via the serial port. Finally add a small delay and increase the frequency of data polling

Result demonstration



(/wiki/File:200px-ESP32-S3-Touch-AMOLED-1.8-demo-13.png)

12_LVGL_AXP2101_ADC_Data

Demo description

- This demo demonstrates power management using the XPower library under LVGL, and provides PWR custom button control for screen on and off actions

Hardware connection

- Connect the development board to the computer

Code analysis

- Screen on/off implementation function

```
void toggleBacklight() {
    USBSerial.println(backlight_on);
    if (backlight_on) {
        for (int i = 255; i >= 0; i--) {
            gfx->Display_Brightness(i);
            delay(3);
        }
    } else {
        for (int i = 0; i <= 255; i++) {
            gfx->Display_Brightness(i);
            delay(3);
        }
    }
    backlight_on = !backlight_on;
}
```

Result demonstration

- Display parameters: chip temperature, charging state, discharging state, standby state, Vbus connected, Vbus condition, charger status, battery voltage, Vbus voltage, system voltage, battery percentage



(/wiki/File:200px-ESP32-S3-Touch-AMOLED-1.8-demo-12.png)

13_LVGL_Widgets

Demo description

- This demo demonstrates the LVGL Widgets example, the frame rate can reach 50~60 frames in the dynamic state, **By optimizing the SH8601 display library to achieve a better and smoother frame rate, which can actually be compared to the scenario with double cache and double acceleration enabled in ESP-IDF environment**

Hardware connection

- Connect the development board to the computer

Code analysis

- **setup**: Responsible for initializing various hardware devices and LVGL graphics library environment
 - Serial port initialization: `USBSerial1.begin(115200)` prepares for serial port debugging
 - I2C bus Initialization: `Wire.begin(IIC_SDA, IIC_SCL);` initializes I2C bus for communication with other I2C devices
 - Expansion chip initialization: Create and initialize the expansion chip `expander`, set the pin mode as the output, and make some initial pin state settings

- Touch controller initialization: Keep trying to initialize the touch controller `FT3168` , if initialization fails, print an error message and wait with a delay, and print a success message if successful
- Graphical display initialization: Initialize the graphical display device `gfx` , set the brightness, and get the width and height of the screen. Then initialize the LVGL, including registering the print callback function for debugging, setting the touch controller's power mode to monitor mode, initializing the display driver and the input device driver. Create and start the LVGL timer, create a label and set the text, and finally call `lv_demo_widgets()` to display the LVGL sample widget
- **loop**
 - `lv_timer_handler()` : This is an important function in the LVGL graphics library, which is used to handle various timer events, animation updates, input processing and other tasks of the graphical interface. Calling this function in each loop ensures smooth operation of the graphical interface and timely response to interactions
 - `delay(5);` : Add a small delay to avoid overoccupying CPU resources

Result demonstration



(/wiki/File:200px-ESP32-S3-Touch-AMOLED-1.8-demo-14.png)



(/wiki/File:197px-ESP32-S3-Touch-AMOLED-1.8-demo-15.png)

14_LVGL_SD_Test

Demo description

- This demo demonstrates using SDMMC to drive a TF card and output its contents to the display

Hardware connection

- Connect the development board to the computer
- Insert the TF card into the board

Code analysis

- **setup**: Responsible for initializing various hardware devices and LVGL graphics library environment
 - Serial port initialization: `USBSerial.begin(115200)` prepares for serial port debugging
 - I2C bus and expansion chip initialization: Initialize the I2C bus, create and initialize the expansion chip, set its pin mode to output, and print the initial and new states
 - Graphics display initialization: Initialize graphics display device, set brightness, initialize LVGL, set display driver and input device driver, create and start LVGL timer
 - TF card Initialization and information display: Set the pins of the TF card and attempt to mount the TF card. If the mount fails, an error message is displayed on the serial port and the screen. If the mount is successful, detect the TF card type and display it, get the TF card size and display it. Then call the `listDir` function to list the contents of the TF card root directory, and display the TF card type, size, and directory list information in the label on the screen
- **listDir**: Recursively lists the files and subdirectories in the specified directory
 - First print the name of the directory being listed. Then open the specified directory, and if the opening fails, an error message will be returned. If it is not a directory that is opened, an error message is also returned. If it is a directory, then traverse the files and subdirectories in it. For subdirectories, recursively call the `listDir` function to list their contents. For files, print the file name and file size. Finally, return all collected information as a string

Result demonstration

TF card control pin description

SPI interface	ESP32-S3
CS (SS)	EXIO7
DI (MOSI)	GPIO1
DO (MISO)	GPIO3
SCK (SCLK)	GPIO2



(/wiki/File:200px-ESP32-S3-Touch-AMOLED-1.8-demo-16.png)

15_ES8311

Demo description

- This demo demonstrates using I2S to drive the ES8311 chip to play the converted binary audio file

Hardware connection

- Connect the development board to the computer

Code analysis

- **es8311_codec_init**: Initializes the ES8311 audio codec
 - Create a handle `es_handle` for the ES8311 codec
 - Configure the clock parameters of ES8311, including the main clock and sampling clock frequency, as well as the clock polarity, etc.
 - Initialize the codec and set the audio resolution to 16 bits
 - Configure sampling frequency
 - Configure microphone-related parameters, such as turning off the microphone, setting volume, and microphone gain
- **setup**: Perform overall initialization settings, including serial port, pins, I2S, and ES8311 codec
 - Initialize serial port for debugging output
 - Set a specific pin as output and set it high
 - Configure I2S bus, set pins, operating mode, sampling rate, data bit width, channel mode, etc.
 - Initialize the I2C bus
 - Call the `es8311_codec_init` function to initialize the ES8311 codec
 - Play a predefined piece of audio data (`canon_pcm`) through the I2S bus

Result demonstration

- Nothing will be shown on the screen if you play the audio file

16_LVGL_Sqprj

Demo description

- This demo demonstrates the use of squareline ui combined with LVGL examples, using the QMI8658 sensor to achieve adaptive screen display orientation. It displays real-time time via WIFI and changes the backlight brightness through touch input in specific areas

Hardware connection

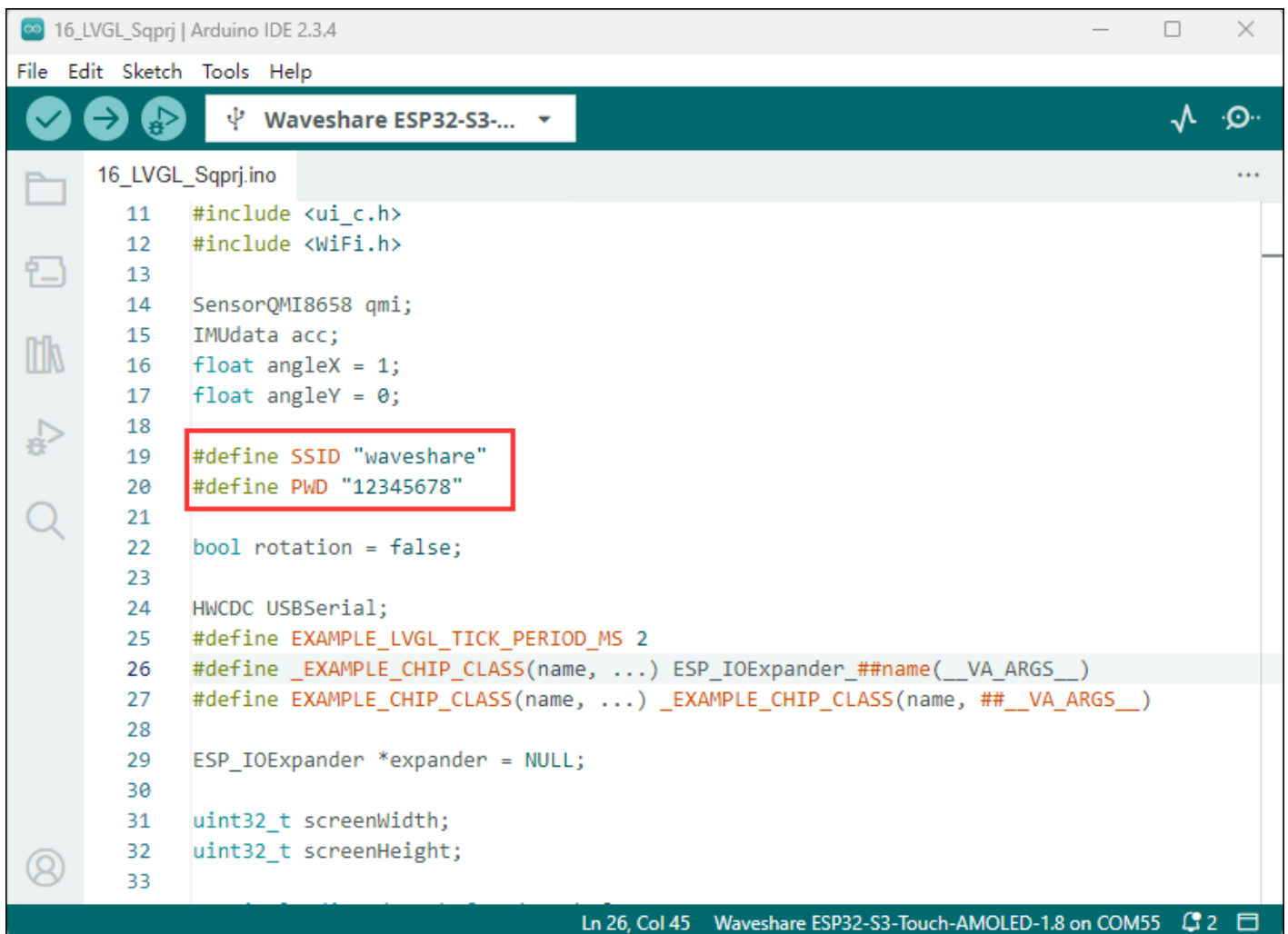
- Connect the development board to the computer

Code analysis

- **setup**
 - Serial port initialization
 - Wi-Fi initialization: Wi-Fi connection and network time configuration
 - I2C communication and expander initialization: Initialize the I2C communication bus, specify the SDA and SCL pins, configure multiple expander pins (pins 0, 1, 2, 6)
 - Sensor initialization: Initialize QMI8658 sensor, initialize touch controller (FT3168 object)
 - Display initialization and related configuration
- **loop()**
 - Interface rotation processing based on sensor data
 - LVGL graphical interface update
 - Display brightness adjustment processing
 - Interface element value update

Code modification

- The image content resources are large and need to be set **Tools -> Partition Scheme -> 16M Flash (3MB APP/9.9MB FATFS)**
- Modify to your available WiFi



```
16_LVGL_Sqprj.ino
11 #include <ui_c.h>
12 #include <WiFi.h>
13
14 SensorQMI8658 qmi;
15 IMUdata acc;
16 float angleX = 1;
17 float angleY = 0;
18
19 #define SSID "waveshare"
20 #define PWD "12345678"
21
22 bool rotation = false;
23
24 HWCDC USBSerial;
25 #define EXAMPLE_LVGL_TICK_PERIOD_MS 2
26 #define _EXAMPLE_CHIP_CLASS(name, ...) ESP_IOExpander_##name(__VA_ARGS__)
27 #define EXAMPLE_CHIP_CLASS(name, ...) _EXAMPLE_CHIP_CLASS(name, ##__VA_ARGS__)
28
29 ESP_IOExpander *expander = NULL;
30
31 uint32_t screenWidth;
32 uint32_t screenHeight;
33
```

Ln 26, Col 45 Waveshare ESP32-S3-Touch-AMOLED-1.8 on COM55 2

(/wiki/File:ESP32-S3-Touch-AMOLED-1.8-demo-17.png)

Result demonstration



(/wiki/File:200px-ESP32-S3-Touch-AMOLED-1.8-demo-23.png)

Working with ESP-IDF

This chapter introduces setting up the ESP-IDF environment setup, including the installation of Visual Studio and the Espressif IDF plugin, program compilation, downloading, and testing of demos, to assist users in mastering the development board and facilitating secondary

development.



(/wiki/File:ESP-IDF-flow-01.png)

Environment Setup

Download and Install Visual Studio

- Open the download page of VScode official website (<https://code.visualstudio.com/download>), choose the corresponding system and system bit to download

Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.



↓ Windows

Windows 10, 11

User Installer

x64

Arm64

System

x64

Arm64

Installer

x64

Arm64

.zip

x64

Arm64

CLI

x64

Arm64



↓ .deb

Debian, Ubuntu

.deb

x64

Arm32

Arm64

.rpm

x64

Arm32

Arm64

.tar.gz

x64

Arm32

Arm64

Snap

Snap Store

CLI

x64

Arm32

Arm64

↓ .rpm

Red Hat, Fedora, SUSE



↓ Mac

macOS 10.15+

.zip

Intel chip

Apple silicon

Universal

CLI

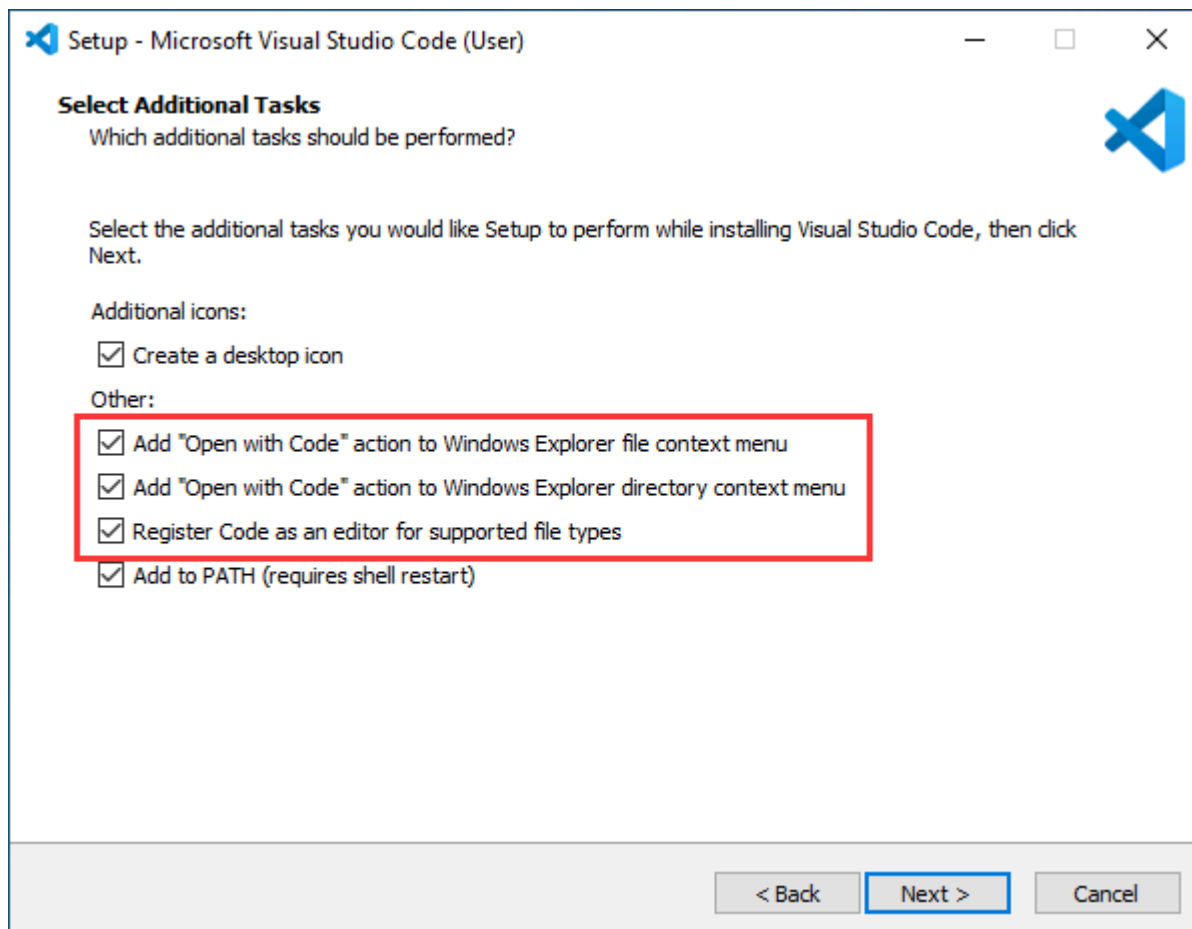
Intel chip

Apple silicon

(/wiki/File:ESP32-S3-AMOLED-1.91-VScode-01.png)

- After running the installation package, the rest can be installed by default, but here for the subsequent experience, it is recommended to check boxes 1, 2, and 3

From Waveshare Wiki: Open-source documentation for hundreds of models. For the latest version of the manual or further support, please contact customer service.



(/wiki/File:ESP32-S3-AMOLED-1.91-VScode-02.png)

- After the first two items are enabled, you can open VSCode directly by right-clicking files or directories, which can improve the subsequent user experience.
- After the third item is enabled, you can select VSCode directly when you choose how to open it.

The environment setup is carried out on the Windows 10 system, Linux and Mac users can access ESP-IDF environment setup (<https://docs.espressif.com/projects/esp-idf/en/v5.1.4/espressif3/get-started/windows-setup.html>) for reference

Install Espressif IDF Plugin

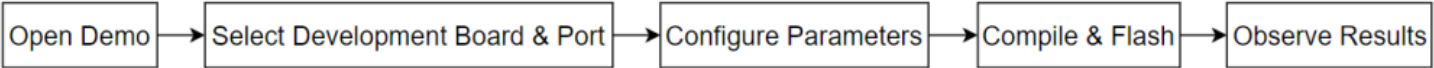
- It is generally recommended to use **Install Online**. If online installation fails due to network factor, use **Install Offline**.

- For more information about how to install the Espressif IDF plugin, see [Install Espressif IDF Plugin \(\)](#)

Run the First ESP-IDF Demo

If you are just getting started with ESP32 and ESP-IDF, and you don't know how to create, compile, flash, and run ESP-IDF ESP32 programs, then please expand and take a look. Hope it can help you! [\[Expand\]](#)

Demos



(/wiki/File:Demo-flow-01.png)

- ESP32-S3-Touch-AMOLED-1.8 Demo

Demo	Basic Description
01_AXP2101	Obtain power-related data through the ported XPowerLib to drive AXP2101
02_PCF85063	Drive pcf85063 for time storage and reading function
03_QMI8658	Use the ported SensorLib to drive qmi8658 to obtain gyroscope-related data
04_SD_MMC	Initialize TF card using sdmmc and read/write the content
05_LVGL_WITH_RAM	Run the LVGL demo by enabling dual caching and DMA acceleration anti-tearing, etc.
06_I2SCodec	Use I2S to drive ES8311 Codec chip to record or play audio

01_AXP2101

Demo description

- This demo demonstrates using ESP-IDF to port XPowerLib, and driving AXP2101 to obtain power-related data through the ported XPowerLib

Hardware connection

- Connect the development board to the computer

Code analysis

- **i2c_init**: Initializes the I2C master in preparation for communication with other devices, such as the PMU
 - Configure I2C parameters, including setting the master device mode, specifying the SDA and SCL pins, enabling the pull-up resistor, and determining the clock frequency
 - Install the I2C driver and apply the configuration to the actual hardware
- **pmu_register_read**: Reads a series of bytes of data from a specific register of the PMU
 - Perform parameter checks to ensure the incoming parameters are valid and avoid invalid read operations
 - Perform I2C operations in two steps, first send the register address to read, then read the data. During the reading process, different processing is carried out according to the length of bytes to be read to ensure accurate reading of the data. At the same time, handle error cases in the I2C communication process and return the corresponding status code so that the upper-layer code can determine if the read operation is successful

Result demonstration

- This demo will not light up the screen
- The serial port monitor displays the parameters: chip temperature, charging state, discharging state, standby state, Vbus connected, Vbus condition, charger status, battery voltage, Vbus voltage, system voltage, battery percentage

```
I (25473) AXP2101: Power Temperature: 32.45°C
I (25473) AXP2101: isCharging: NO
I (25473) AXP2101: isDischarge: NO
I (25473) AXP2101: isStandby: YES
I (25473) AXP2101: isVbusIn: YES
I (25483) AXP2101: isVbusGood: YES
I (25483) AXP2101: Charger Status: charge done
I (25493) AXP2101: getBattVoltage: 4131 mV
I (25493) AXP2101: getVbusVoltage: 5243 mV
I (25503) AXP2101: getSystemVoltage: 4372 mV
I (25503) AXP2101: getBatteryPercent: 100 %
```

(/wiki/File:ESP32-S3-Touch-AMOLED-1.8-demo-21.png)

02_PCF85063

Demo description

- This demo uses a simple way to drive the PCF85063 for time storage and reading functionality

Hardware connection

- Connect the development board to the computer

Code analysis

- **i2c_master_init**
 - Define the I2C configuration structure `conf`, set the master device mode, SDA and SCL pins, pull-up resistor, and clock frequency
 - Use the `i2c_param_config` function to configure I2C parameters. If the configuration fails, an error log is recorded and an error code is returned
 - Use the `i2c_driver_install` function to install the I2C driver, apply the configuration to the actual hardware, and return the result
- **rtc_get_time**
 - Define a byte array named `data` with a length of 7 for storing read time data
 - Call the `rtc_read_reg` function to read 7 bytes of time data starting from the specific register address (0x04) of the RTC chip. If the reading fails, an error log is recorded and an error code is returned
 - Process the time data read, separately extract the seconds, minutes, hours, days, weeks, months, and years, and convert BCD to decimal
 - Use `ESP_LOGI` to output the formatted current time

Result demonstration

- This demo will not light up the screen
- The serial port monitor prints time information

```
I (302) sleep: Enable automatic switching of GPIO sleep configuration
I (309) main_task: Started on CPU0
I (319) main_task: Calling app_main()
I (319) PCF85063: Time set successfully
I (319) PCF85063: Current Time: 12:30:45 27/09/2024
I (1329) PCF85063: Current Time: 12:30:46 27/09/2024
I (2329) PCF85063: Current Time: 12:30:47 27/09/2024
I (3329) PCF85063: Current Time: 12:30:48 27/09/2024
I (4329) PCF85063: Current Time: 12:30:49 27/09/2024
I (5329) PCF85063: Current Time: 12:30:50 27/09/2024
I (6329) PCF85063: Current Time: 12:30:51 27/09/2024
I (7329) PCF85063: Current Time: 12:30:52 27/09/2024
I (8329) PCF85063: Current Time: 12:30:53 27/09/2024
I (9329) PCF85063: Current Time: 12:30:54 27/09/2024
I (10329) PCF85063: Current Time: 12:30:55 27/09/2024
I (11329) PCF85063: Current Time: 12:30:56 27/09/2024
```

(/wiki/File:ESP32-S3-Touch-AMOLED-1.8-demo-19.png)

03_QMI8658

Demo description

This demo demonstrates using ESP-IDF to port SensorLib, then use the ported SensorLib to drive qmi8658 to obtain gyroscope-related data

Hardware connection

- Connect the development board to the computer

Code analysis

- **setup_sensor**: Set and initialize the environment and parameters required to communicate with the QMI8658 sensor
 - Initialize I2C communication to ensure that the connection channel to the sensor is established
 - Initialize the sensor, check if the sensor is properly connected
 - Configure the sensor's accelerometer and gyroscope parameters to meet specific application needs
 - Enable the sensor to start collecting data
- **read_sensor_data**: Read and process data from the QMI8658 sensor in a continuous loop and output the results
 - In the loop, continuously check if the sensor data is ready
 - When the data is ready, the accelerometer, gyroscope, timestamp, and temperature data are read and logged out
 - Handle the failure of data reading by recording an error log for troubleshooting
 - Control the execution speed of the loop through delay to avoid excessive consumption of system resources

Result demonstration

- This demo will not light up the screen
- The serial port monitor prints sensor data

```
I (17160) QMI8658: Timestamp: 14768, Temperature: 32.71 *C
I (17260) QMI8658: ACCEL: 0.054932, 0.158325, -1.044312
I (17260) QMI8658: GYRO: -3.953125, 0.964844, 0.037109
I (17260) QMI8658: Timestamp: 14858, Temperature: 32.70 *C
I (17360) QMI8658: ACCEL: 0.054932, 0.156494, -1.044800
I (17360) QMI8658: GYRO: -1.871094, 0.837891, 0.201172
I (17360) QMI8658: Timestamp: 14947, Temperature: 32.70 *C
I (17460) QMI8658: ACCEL: 0.054932, 0.160156, -1.039917
I (17460) QMI8658: GYRO: -3.007812, 1.027344, -0.070312
I (17460) QMI8658: Timestamp: 15037, Temperature: 32.71 *C
I (17560) QMI8658: ACCEL: 0.054932, 0.157349, -1.046387
I (17560) QMI8658: GYRO: -3.472656, 1.068359, 0.222656
I (17560) QMI8658: Timestamp: 15126, Temperature: 32.71 *C
I (17660) QMI8658: ACCEL: 0.054932, 0.157959, -1.042236
I (17660) QMI8658: GYRO: -2.640625, 0.720703, 0.138672
I (17660) QMI8658: Timestamp: 15216, Temperature: 32.73 *C
```

(/wiki/File:ESP32-S3-Touch-AMOLED-1.8-demo-20.png)

04_SD_MMC

Demo description

This demo demonstrates initializing TF card using sdmmc and read/write the content

Hardware connection

- Connect the development board to the computer

Code analysis

- **esp_vfs_fat_sdmmc_mount**: Mount the FAT file system of the TF card to the specified mount point and establish a file access interface for the TF card
 - Prepare the mount configuration, determine whether to format and other parameters when the mount fails
 - Initialize the SDMMC host configuration, including settings such as frequency. For specific chips, power control initialization may also be required
 - Configure SDMMC card slot parameters such as bus width and GPIO pin settings
 - Call the function to perform the mounting operation, try to initialize the TF card and mount the file system. If successful, you can get the TF card information; if it fails, an error code is returned for error handling
- **s_example_write_file**: Write data to a file in a specified path to create and update file contents on the TF card
 - Open the file at the specified path in write mode, and if it fails to open, log the error and return the error code
 - Use the standard C library function `fprintf` to write data to a file

- Close the file and record a log indicating that the file has been written. If there are issues while opening or writing, it will affect the creation of the file and the saving of data

Result demonstration

- This demo will not light up the screen
- The serial port monitor prints the TF card information

```
I (590) example: Filesystem mounted
Name: SD
Type: SDHC/SDXC
Speed: 20.00 MHz (limit: 20.00 MHz)
Size: 15070MB
CSD: ver=2, sector_size=512, capacity=30863360 read_bl_len=9
SSR: bus_width=1
I (600) example: Opening file /sdcard/hello.txt
I (620) example: File written
I (810) example: Renaming file /sdcard/hello.txt to /sdcard/foo.txt
I (810) example: Reading file /sdcard/foo.txt
I (810) example: Read from file: 'Hello SD!'
I (810) example: Opening file /sdcard/nihao.txt
I (830) example: File written
I (830) example: Reading file /sdcard/nihao.txt
I (840) example: Read from file: 'Nihao SD!'
I (840) example: Card unmounted
I (840) main_task: Returned from app_main()
```

(/wiki/File:ESP32-S3-Touch-AMOLED-1.8-demo-22.png)

05_LVGL_WITH_RAM

Demo description

- This example shows LVGL demo, which can run LVGL demo by enabling dual caching, enabling DMA acceleration and anti-tearing, etc., and run dynamic graphics and texts smoothly, **the frame rate can reach 200~300 frames**

Hardware connection

- Connect the development board to the computer

Code analysis

- **app_main**: The initialization and startup process of the entire application
 - Hardware initialization: Configure and initialize hardware devices related to LCD display and touch input (if available), such as GPIO pins, SPI bus, I2C bus, LCD panel driver, and touch controller driver, etc.

- LVGL initialization: Initialize the LVGL library, allocate drawing buffers, register display driver and touch input device driver (if available), install timers, and create mutexes and start LVGL tasks
- Example display: Finally show LVGL examples, such as widget examples, etc.
- **example_lvgl_port_task**: LVGL task function, responsible for handling LVGL regular updates and task delay control, ensuring the smooth operation of the LVGL interface
 - Timed update: The lv_timer_handler is called to handle the timer event of LVGL to update the interface state
 - Delay control: Determine the task's delay time based on the results of timer processing to avoid excessive CPU resource usage and ensure timely response to LVGL events

Result demonstration



(/wiki/File:200px-ESP32-S3-Touch-AMOLED-1.8-demo-14.png)



(/wiki/File:197px-ESP32-S3-Touch-AMOLED-1.8-demo-15.png)

06_I2SCodec

Demo description

This demo demonstrates using the I2S to drive the ES8311 Codec chip to capture or play audio

Hardware connection

- Connect the development board to the computer

Code analysis

- **app_main**: The initialization and startup process of the entire application
 - Initialize the GPIO, set a specific pin to output mode and control its level state
 - Print startup information, provide initial status feedback of program running LVGL task
 - Initialize I2S peripherals, configure I2S channels and related hardware parameters, and ensure the transmission of audio data
 - Initialize ES8311 codec, configure the I2C bus and set the codec parameters to prepare for audio processing
 - Create corresponding tasks according to different operating modes to realize music playback or microphone echo function
- **es8311_codec_init**: Responsible for initializing I2C peripherals and configuring the ES8311 audio codec to ensure that the codec can work normally
 - Initialize the I2C bus, including setting parameters such as pins, modes, clock speed, etc., and install an I2C driver to establish the foundation for communication with the ES8311
 - Create a handle to the ES8311 codec for subsequent configuration operations
 - Configure the clock parameters of the ES8311, such as the reversal state of the host and slave clocks, the host clock source, frequency, and sampling frequency
 - Initialize the ES8311 codec and set parameters such as audio resolution
 - Set the sampling frequency, volume, and microphone parameters (as needed) to meet specific audio processing needs

Result demonstration

- This demo will not light up the screen, only play the audio
- The serial port monitor can view the serial port information

```

I (435) gpio: GPIO[46]| InputEn: 0| OutputEn: 1| OpenDrain: 0| Pullup: 0| Pulldown: 0| Intr:0
I (435) gpio: GPIO[46]| InputEn: 0| OutputEn: 1| OpenDrain: 0| Pullup: 0| Pulldown: 0| Intr:0
i2s es8311 codec example start
i2s es8311 codec example start
-----
I (445) i2s_es8311: i2s driver init success
-----
I (445) i2s_es8311: i2s driver init success
I (475) ES8311: ES8311 in Slave mode and I2S format
I (475) ES8311: ES8311 in Slave mode and I2S format
I (485) i2s_es8311: es8311 codec init success
I (485) i2s_es8311: es8311 codec init success
I (485) main_task: Returned from app_main()
I (10405) i2s_es8311: [music] i2s music played, 634240 bytes are written.
I (21325) i2s_es8311: [music] i2s music played, 640000 bytes are written.
I (32225) i2s_es8311: [music] i2s music played, 640000 bytes are written.

```

(/wiki/File:ESP32-S3-Touch-AMOLED-1.8-demo-18.png)

Custom audio

Users can customize audio playback segments by following the steps below (some programming knowledge is required)

- Select any format of music you want to play (e.g. a.mp3)
- Install **ffmpeg** tool
- Use the command line to view the details of an audio file, including the file's encoding format, duration, bitrate, audio stream details, etc.
 - **ffprobe a.mp3**
- Cut your music because there isn't enough room for the whole piece of music
 - **ffmpeg -i a.mp3 -ss 00:00:00 -t 00:00:20 a_cut.mp3**
- Convert music format to .pcm
 - **ffmpeg -i a_cut.mp3 -f s16le -ar 16000 -ac 1 -acodec pcm_s16le a_cut.pcm**
- **Move** the cropped **a_cut.pcm** audio file to **main** directory
- **Replace** canon.pcm in main\CMakeList.txt with a_cut.pcm
- **Replace** _binary_canon_pcm_start and _binary_canon_pcm_end in main\i2s_es8311_example.c with _binary_a_cut_pcm_start and _binary_a_cut_pcm_end
- Compile and flash

Flash Firmware Flashing and Erasing

- The current demo provides test firmware, which can be used to test whether the onboard device functions properly by directly flashing the test firmware
- Binary file path:

```
..\ESP32-S3-Touch-AMOLED-1.8-Demo\Firmware
```


Resources

Schematic Diagram

- ESP32-S3-Touch-AMOLED-1.8 Schematic diagram (<https://files.waveshare.com/wiki/ESP32-S3-Touch-AMOLED-1.8/ESP32-S3-Touch-AMOLED-1.8.pdf>)

Project diagram

- ESP32-S3-Touch-AMOLED-1.8 3D diagram (<https://files.waveshare.com/wiki/ESP32-S3-Touch-AMOLED-1.8/ESP32-S3-Touch-AMOLED-1.8-3D.zip>)

Demo

- ESP32-S3-Touch-AMOLED-1.8 Demo (<https://files.waveshare.com/wiki/ESP32-S3-Touch-AMOLED-1.8/ESP32-S3-Touch-AMOLED-1.8-Demo.zip>)

Datasheets

ESP32-S3

- ESP32-S3 Series Datasheet (https://files.waveshare.com/wiki/common/Esp32-s3_datasheet_en.pdf)
- ESP32-S3 Technical Reference Manual (https://files.waveshare.com/wiki/common/Esp32-s3_technical_reference_manual_en.pdf)

Other Components

- QMI8658 Datasheet (<https://files.waveshare.com/wiki/common/QMI8658C.pdf>)
- PCF85063 Datasheet (<https://files.waveshare.com/wiki/common/PCF85063A.pdf>)
- AXP2101 Datasheet (https://files.waveshare.com/wiki/common/X-power-AXP2101_SWcharge_V1.0.pdf)
- ES8311 Datasheet (<https://files.waveshare.com/wiki/common/ES8311.DS.pdf>)
- ES8311 User Guide (<https://files.waveshare.com/wiki/common/ES8311.user.Guide.pdf>)
- SH8601 Datasheet (https://files.waveshare.com/wiki/common/SH8601A0_DataSheet_Preliminary_V0.0_UCS_191107_1_.pdf)

- FT3168 Datasheet (<https://files.waveshare.com/wiki/common/FT3168.pdf>)

Softwares

- Zimo221 Chinese character modulation software (<https://files.waveshare.com/wiki/common/Zimo221.7z>)
- Image2Lcd image modulation software (<https://files.waveshare.com/wiki/common/Image2Lcd2.9.zip>)
- flash_download_tool_3.9.7 (https://files.waveshare.com/wiki/common/flash_download_tool_3.9.7_1.zip)

Other Resource Links

- Image modulation tutorial ()
- Font library modulation tutorial ()
- MicroPython official documentation (<https://docs.micropython.org/en/latest/>)
- ESP32 Arduino Core's documentation (<https://docs.espressif.com/projects/arduino-esp32/en/latest/index.html>)
- arduino-esp32 (<https://github.com/espressif/arduino-esp32>)
- ESP-IDF (<https://github.com/espressif/esp-idf>)

Project Resources

This section features third - party project resources. We merely provide links and bear no responsibility for content updates or maintenance. Thank you for your understanding.

Volos Projects

①Make A Step Counter Pedometer

- Youtube: <https://youtu.be/ySCa7F-0yzY?si=luRPWQzthT-BvaGd> (<https://youtu.be/ySCa7F-0yzY?si=luRPWQzthT-BvaGd>)
- Github: <https://github.com/VolosR/stepCounter> (<https://github.com/VolosR/stepCounter>)

② PocketClock example and some install tips!

- Youtube: <https://www.youtube.com/watch?v=AbDc1Px0TsM> (<https://www.youtube.com/watch?v=AbDc1Px0TsM>)
- Github: <https://github.com/VolosR/pocketClock> (<https://github.com/VolosR/pocketClock>)

③ How to Open and Customize UI in SquareLine Studio (Stopwatch Example)

- Youtube: <https://www.youtube.com/watch?v=B7Sh63ggaWw> (<https://www.youtube.com/watch?v=B7Sh63ggaWw>)
- Github: <https://github.com/VolosR/stopwatchAmoled> (<https://github.com/VolosR/stopwatchAmoled>)

nishad2m8-Smartwatch UI with LVGL & EEZ Studio | 1.8" Display + Step Counter

- Youtube : <https://www.youtube.com/watch?v=8sJgvdSTEjQ> (<https://www.youtube.com/watch?v=8sJgvdSTEjQ>)

- Github : <https://github.com/nishad2m8/Squareline-OBP> (<https://github.com/nishad2m8/Squareline-OBP>)

FAQ

Question:I want more driver library support?

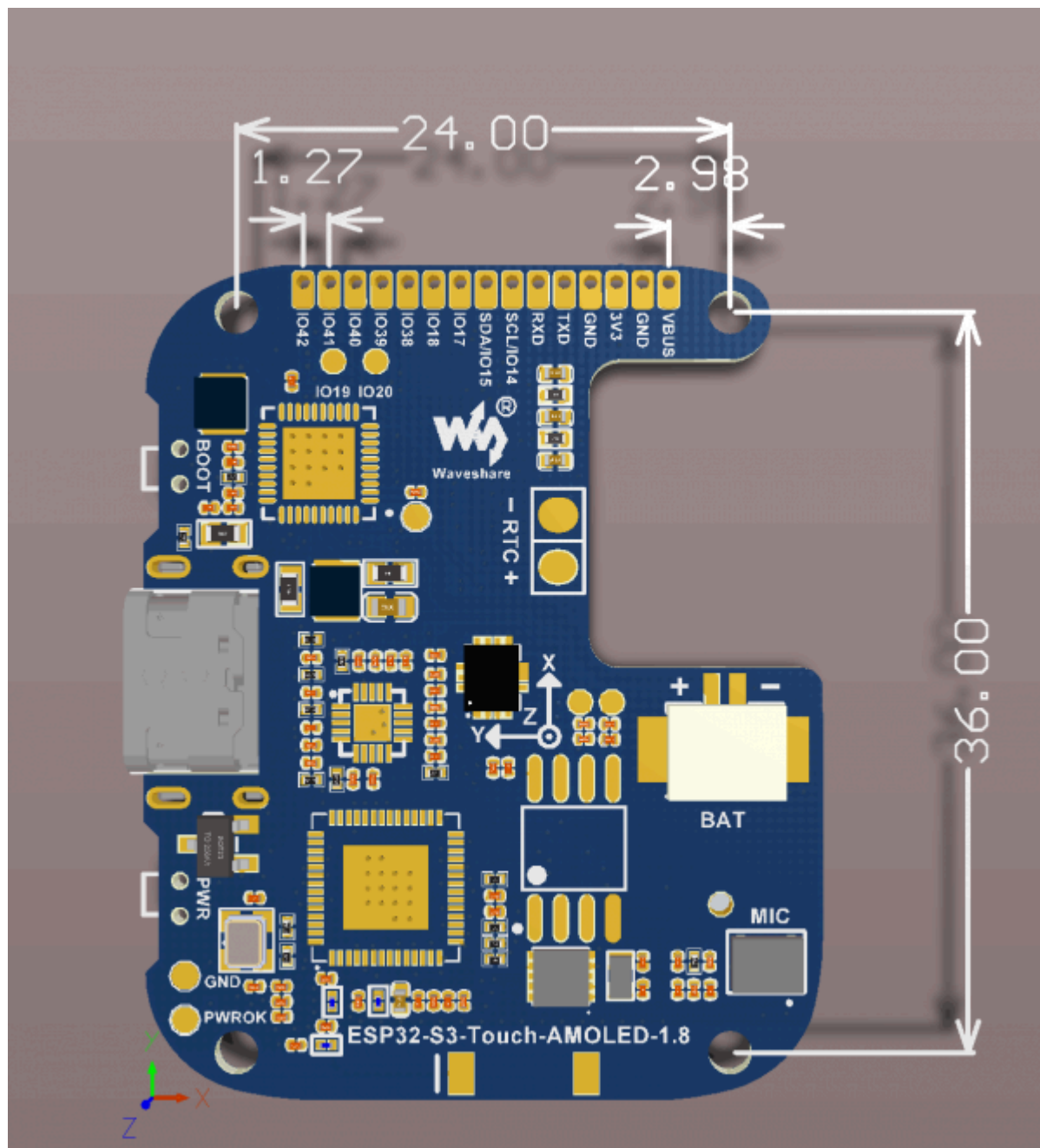
Answer:

You can subscribe to the repository, just open an issue, explain your requirements clearly, engineers will be the first time to evaluate the requirements, if there is any progress and update the repository, and you can also joke about the product, share your experience, and put forward your ideas. ESP32-display-support (<https://github.com/waveshareteam/ESP32-display-support/tree/master>)

Question: What is the pin size of this product, and what is the copper pillar size?

Answer:

The reserved pad spacing of this product is 1.27mm, and it is compatible with copper pillars of M2 size.



(/wiki/File:600px-ESP32-S3-Touch-AMOLED-1.8-241206_01.png)

Question: What platforms does the AI large model example of speech dialog support?

Answer:

The product is equipped with ES8311 audio codec chip, onboard speaker, and SMD microphone. And it can achieve voice dialogue function;

At present, the access to various platforms has been verified, and mainstream platforms

such as Doubao, ERNIE Bot , and ChatGPT are supported. The demos will gradually be opened up to ESP32-AIChats (<https://github.com/waveshareteam/ESP32-AIChats>)

Question: How is the heat generation of this product, and will it affect the display?

Answer:

The measured temperature is at room temperature 20°C, the wifi STA and AP modes are turned on, and the battery is in the charging state. The test lasts for half an hour, and the highest temperature read through AXP2101 temperature data is 46 °C. During the normal discharge process, the temperature will decrease by 3-4 °C. If WiFi/Bluetooth function is not turned on, it can maintain a stable temperature of around 36 °C

This AMOLED screen can withstand high temperatures; at 40 to 60 degrees Celsius, it will not affect the screen display or touch function. In high temperature and high humidity conditions, there may be some polarization, which is within the normal range.

For this product, it is recommended to use the low-power operation mode of ESP32-S3 to complete the application in some scenarios.

Question: Why does the flashing fail?

Answer:

1. When the serial port is occupied, the flashing will fail. Close the serial port monitor and try to flash again
2. When the ESP32 program crashes, the flashing will fail, and the development module needs to be completely powered off, hold down BOOT and power it on again to enter the strong download mode and then flash it. It will not automatically exit the download mode after flashing, so you need to power off and restart again

Note: To power on this product again, press and hold the PWR button for at least 6 seconds, and then press the PWR button again

Question: Why did the compilation fail?

Answer:

When the project fails to be compiled, check the development environment.

1. Arduino IDE compiling the project shows the situation in the image, you need to modify **Partition Scheme-> 16M Flash(3MB APP/9.9MB FATFS)**.

```
Sketch uses 1404340 bytes (107%) of program storage space. Maximum is 1310720 bytes.  
Global variables use 96644 bytes (29%) of dynamic memory, leaving 231036 bytes for local variables. Maximum is 327680 bytes.  
Sketch too big; see https://support.arduino.cc/hc/en-us/articles/360013825179 for tips on reducing it.  
text section exceeds available space in board  
  
Compilation error: text section exceeds available space in board
```

(/wiki/File:ESP32-S3-Touch-AMOLED-1.8-FAQ_241227_01.png)

2. When the ESP-IDF project compilation fails, delete the build, managed_components, and dependencies.lock, and then compile again. If the error persists, check whether the ESP-IDF version and the CMakeList.txt in the project directory are correctly recognized

Question: Why does the module keep resetting?

Answer:

In this case, due to the crash of the program, the USB cannot work normally, you can press and hold the BOOT button and then power on, at this time, the module can enter the download mode to flash the firmware (program) to solve the situation.

Question: How to deal with the first compilation of the program being extremely slow?

Answer:

In the Arduino IDE, the first compilation loads and compiles all the dependencies, so it's normal for the first compilation task to be time-consuming. Just wait patiently

Question: How to handle the display "waiting for download..." on the serial port after successfully ESP-IDF flashing?

Answer:

Power it on again

Question: How to measure the battery voltage of this product?

Answer:

This product has an on-board AXP2101, which can measure all kinds of battery data, such as: chip temperature, whether it is charged, whether it is connected to the power supply, charging status, battery voltage, charging voltage, battery power, etc.

Question: This product measures the percentage of battery remaining capacity. Why does the result fluctuate greatly?

Answer:

The battery capacity measurement of the AXP2101 is based on battery voltage, but battery capacity does not vary linearly, so large percentage fluctuations may occur during measurement. Especially in the case of plugging and unplugging chargers, load changes, or battery aging, the fluctuations become more noticeable. The measured percentages are for reference only, and the actual voltage is used as a measure of capacity.

Question: Can you provide the 3D file for the casing?

Answer:

It is not provided.

Question: Can both buttons of this product be programmed for use?

Answer:

Yes, the two buttons of this product are BOOT and PWR. The functions are as follows:
1. BOOT: Press and then power on, and the development board enters download mode (commonly used when the program freezes or the USB GPIO is occupied); under normal working conditions, the GPIO0 can detect the high and low level of the button to determine the action, and the low level is pressed, which can recognize click, double-click, multi-click, and long-press actions.

2.PWR: In the power-on state, press and hold for 6s to power off, in the power-off state (power off to charge the battery), click to power on; Under normal working conditions, the action can be judged by the high and low levels of the EXIO4 detection buttons of the extended IO, and the high level is pressed, which can identify the actions of single click, double click, multiple click and long press (long press should not exceed 6s, otherwise the power will be turned off).

Question: Does this product support Bluetooth?

Answer:

The core chip of this product is ESP32-S3R8, which supports wireless communication functions such as 2.4GHz WiFi and Bluetooth 5 (LE), and has an onboard SMD antenna, and the antenna gain has been adjusted to the optimum, which can support the use of Bluetooth 5 (LE) and WiFi. For details, please refer to ESP32-S3 datasheet (https://files.waveshare.com/wiki/common/Esp32-s3_datasheet_en.pdf)

Question: Does this product support Chinese display?

Answer:

This product uses ESP32-S3R8 as the core, connects the screen via QSPI protocol, fully supports Chinese display, but requires meeting the following conditions:

1. Character encoding support is required, and strings that support UNICODE-8 encoding can be rendered to the screen
2. Font library support
3. Display library support, LVGL or Arduino_GFX are supported

Question: What is the maximum brightness of the screen of this product? How to control the screen brightness?

Answer:

350nit;

Write 0x00 (darkest, backlight off) ~ 0xFF (brightest) to the 0x51 via QSPI to control the brightness.

The Arduino_GFX library in Arduino encapsulates custom functions that control backlight via `gfx->Display_Brightness(i)`, where `i` has values from 0 (darkest, backlight off) to 255 (brightest).

Question: What chip is used for the screen of this product?

Answer:

Screen display chip: SH8601;

Screen touch chip: FT3168

Question: What is this product developed with?

Answer:

ESP32-S3-Touch-AMOLED-1.8 is based on the ESP32-S3R8 chip, supports ESP-IDF and its ecosystem, and can be developed using ESP-IDF or arduino-esp32 Core. ESP-IDF can be used with Vscode, and arduino-esp32 can be used with Arduino IDE or PlatformIO.

Question: Is this product waterproof?

Answer:

It is not waterproof and should be kept dry when used

Question: What size is this battery?

Answer:

The recommended battery specification is 3.85*24*28 400mAh, which can be used for about 1 hour in the regular full-light state, 3~4 hours when the screen backlight is turned off, and about 6 hours in full low-power scenarios. The supplied battery (China only) is

Question: Why do the LVGL examples run less smoothly on the Arduino IDE compared to ESP-IDF?

Answer:

The upper driver of the GFX Arduino LVGL is the Arduino TFT library, and the ability to read and write images accelerated by DMA is limited in this scenario. The LVGL examples provided by ESP-IDF enable double buffering and anti-tearing optimization operations, so the results are better.

In the Arduino IDE development environment, we also provide a native `esp_lcd_panel` driver mode, which is significantly improved compared with the screen refresh in the Arduino TFT scenario, but the effect is not as good as the ESP-IDF example.

When using, please change **LV_COLOR_16_SWAP** in "lvgl_conf.h" under Arduino IDE libraries to 1

Arduino_ESP_LCD_LVGL (https://files.waveshare.com/wiki/ESP32-S3-Touch-AMOLED-1.8/Arduino_ESP_LCD_LVGL.zip)

Question: How to check the COM port I use?

Answer:

Windows system:

- ①View through Device Manager: Press the Windows + R keys to open the "Run" dialog box; Input `devmgmt.msc` and press Enter to open Device Manager; Expand the "Port (COM and LPT)" section, here it will list all the COM ports and their current status.
- ②Use the command prompt to view: Open the Command Prompt (CMD); enter the mode command, which will display status information for all COM ports.
- ③Check hardware connections: If you have already connected external devices to the COM port, the device usually occupies a port number, which can be determined by checking the connected hardware.

Linux system:

- ①Use the `dmesg` command to view: Open the terminal.
- ①Use the `ls` command to view: Enter `ls /dev/ttyS*` or `ls /dev/ttyUSB*` to list all serial port

devices.

③Use the setserial command to view: Enter `setserial -g /dev/ttyS*` to view the configuration information of all serial port devices.

Question: Why is there no output from the monitor after running the code, even though the code is correct and successfully flashed?

Answer:

The Type-C flashing and debugging port of ESP32-S3-Touch-AMOLED-1.8 is directly output from ESP32-S3 USB. In the Arduino IDE development environment, it supports `printf` function for printing and output. If you want to support Serial function for printing and output, you need to enable USB CDC On Boot function or HWCDC declaration. For this, please refer to the demonstration in the demo. ESP-IDF usually uses `ESP_LOGD` and `ESP_ERROR_CHECK` for printing output.

Question: Can you help me with the code? Can you help me modify the code?

Answer:

This product is positioned as a development board and is not a finished product. Regarding the product ecosystem, ESP32 is a core product with a mature ecosystem and a friendly development environment. We do not assist in modifying the code. As creators and geeks, please use your DIY abilities. If you have any questions, please feel free to ask our engineers for answers.

If you think our products are good, and you want to customize hardware, casings, software,

etc. in batches, please contact business sales team



(/wiki/File:ESP32-S3-Touch-LCD-4.3B-details-13.png)

Question: How to use SquareLine Studio to design interfaces?

Answer:

- Refer to SquareLine Studio tutorial ()

Support

Technical Support

If you need technical support or have any feedback/review, please click the **Submit Now** button to submit a ticket, Our support team will check and reply to you within 1 to 2 working days. Please be patient as we make every effort to help you to resolve the issue.

Working Time: 9 AM - 6 PM GMT+8 (Monday to Friday)

[Submit Now \(\)](#)